

Here is a **refined and structured** implementation of a **CORBA-based distributed calculator** that demonstrates **object brokering**. This version improves **code clarity**, **error handling**, and **deployment steps** for easy execution.

Steps to Implement a Distributed Calculator using CORBA

1. **Define the IDL (Interface Definition Language) file** for the Calculator.
 2. **Generate CORBA Stubs and Skeletons** using IDL Compiler.
 3. **Implement the CORBA Server** to register the remote Calculator object.
 4. **Implement the CORBA Client** to invoke the remote object's methods.
 5. **Run the ORB (Object Request Broker) Naming Service** and execute the Client-Server.
-

1 Define the IDL File (Calculator.idl)

The **IDL (Interface Definition Language)** describes the Calculator interface:

```
module CalculatorApp {
    interface Calculator {
        float add(in float a, in float b);
        float subtract(in float a, in float b);
        float multiply(in float a, in float b);
        float divide(in float a, in float b) raises(DivisionByZero);
    };

    exception DivisionByZero {};
};
```

Save this as Calculator.idl.

The **DivisionByZero exception** ensures safe division operations.

2 Generate CORBA Stubs and Skeletons

After writing `Calculator.idl`, compile it using:

```
idlj -fall Calculator.idl
```

This generates **CalculatorApp** package with helper classes:

- `Calculator.java`
- `CalculatorHelper.java`
- `CalculatorPOA.java` (for server-side implementation)

3 Implement the CORBA Server (CalculatorServer.java)

The server creates a CORBA object and binds it to the ORB's naming service.

```
import CalculatorApp.*;
import org.omg.CORBA.*;
import org.omg.CosNaming.*;
import org.omg.PortableServer.*;

class CalculatorImpl extends CalculatorPOA {
    private ORB orb;

    public void setORB(ORB orb_val) {
        orb = orb_val;
    }

    public float add(float a, float b) {
        return a + b;
    }

    public float subtract(float a, float b) {
        return a - b;
    }

    public float multiply(float a, float b) {
        return a * b;
    }

    public float divide(float a, float b) throws DivisionByZero {
        if (b == 0) throw new DivisionByZero();
        return a / b;
    }
}

public class CalculatorServer {
    public static void main(String args[]) {
        try {
            // Initialize ORB
            ORB orb = ORB.init(args, null);

            // Get reference to RootPOA and activate POA Manager
            POA rootpoa =
POAHelper.narrow(orb.resolve_initial_references("RootPOA"));
            rootpoa.the_POAManager().activate();

            // Create servant and register it with ORB
            CalculatorImpl calculator = new CalculatorImpl();
            calculator.setORB(orb);
            org.omg.CORBA.Object ref =
rootpoa.servant_to_reference(calculator);
            Calculator href = CalculatorHelper.narrow(ref);

            // Bind object to Naming Service
            org.omg.CORBA.Object objRef =
orb.resolve_initial_references("NameService");
            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);
            NameComponent path[] = ncRef.to_name("Calculator");
```

```

        ncRef.rebind(path, href);

        System.out.println("    Calculator Server is running...");
        orb.run();
    } catch (Exception e) {
        System.err.println("    Server exception: " + e.getMessage());
        e.printStackTrace();
    }
}
}
}

```

Improvements:

Exception Handling added for robustness.

Naming Service Binding ensures clients can locate the Calculator.

ORB Activation and **POA Management** added for smooth operation.

4 Implement the CORBA Client (CalculatorClient.java)

The client connects to the Naming Service, locates the Calculator, and invokes remote methods.

```

import CalculatorApp.*;
import org.omg.CORBA.*;
import org.omg.CosNaming.*;

public class CalculatorClient {
    public static void main(String args[]) {
        try {
            // Initialize ORB
            ORB orb = ORB.init(args, null);

            // Get reference to Naming Service
            org.omg.CORBA.Object objRef =
            orb.resolve_initial_references("NameService");
            NamingContextExt ncRef = NamingContextExtHelper.narrow(objRef);

            // Resolve Calculator object
            Calculator calculator =
            CalculatorHelper.narrow(ncRef.resolve_str("Calculator"));

            // Perform operations
            System.out.println("Addition (10 + 5): " + calculator.add(10,
5));
            System.out.println("Subtraction (10 - 5): " +
calculator.subtract(10, 5));
            System.out.println("Multiplication (10 * 5): " +
calculator.multiply(10, 5));

            try {
                System.out.println("Division (10 / 2): " +
calculator.divide(10, 2));
                System.out.println("Division (10 / 0): " +
calculator.divide(10, 0)); // Should raise exception
            } catch (DivisionByZero e) {

```

```

        System.out.println("⚠ Error: Division by zero is not
allowed!");
    }

    } catch (Exception e) {
        System.err.println("    Client exception: " + e.getMessage());
        e.printStackTrace();
    }
}
}

```

Improvements:

Handles Division by Zero Exception.

User-friendly error messages for better debugging.

ORB Initialization and Service Resolution Optimized.

5 Compile and Run the CORBA Application

Step 1: Compile the IDL file

```
idlj -fall Calculator.idl
```

Step 2: Compile Java files

```
javac CalculatorServer.java CalculatorClient.java CalculatorApp/*.java
```

Step 3: Start the ORB Naming Service

```
tnameserv -ORBInitialPort 1050 &
```

This starts the CORBA Naming Service on port 1050.

Step 4: Run the CORBA Server

```
java CalculatorServer -ORBInitialPort 1050
```

You should see:

```
Calculator Server is running...
```

Step 5: Run the CORBA Client

```
java CalculatorClient -ORBInitialPort 1050
```

Expected Output:

```

Addition (10 + 5): 15.0
Subtraction (10 - 5): 5.0
Multiplication (10 * 5): 50.0

```

Division (10 / 2): 5.0

△ Error: Division by zero is not allowed!

Summary of Refinements

Proper exception handling (handles division by zero).

Improved CORBA Naming Service binding for better connectivity.

Optimized CORBA object references for seamless communication.

Structured logging and error messages for clarity.

This **distributed CORBA-based Calculator** successfully demonstrates **Object Brokering** in a real-world setup.

Need further improvements? Let me know!

What is CORBA (Common Object Request Broker Architecture)?

CORBA (Common Object Request Broker Architecture) is a **distributed computing framework** developed by **OMG (Object Management Group)** that enables applications to **communicate with each other** over a network, **regardless of programming language or operating system**.

CORBA allows different applications written in **Java, C++, Python, etc.** to communicate seamlessly.

It is based on the concept of **Object Brokering**, where an **Object Request Broker (ORB)** helps clients and servers communicate transparently.

Key Features of CORBA

Language Independence: Supports multiple languages (**Java, C++, Python, etc.**).

Platform Independence: Works across different operating systems (**Windows, Linux, macOS**).

Object-Oriented Communication: Enables remote method calls on distributed objects.

Interoperability: Applications written in different languages can talk to each other.

Scalability: Used in **enterprise-level** applications and distributed systems.

How CORBA Works?

CORBA follows a **client-server model** where the **Object Request Broker (ORB)** acts as a **middleman** that enables communication.

1 Client:

- The client invokes a method on a **remote object**.
- The **Stub (proxy)** translates the request and sends it to the ORB.

2 ORB (Object Request Broker):

- The **ORB** receives the request and forwards it to the correct **server object**.
- The ORB provides **location transparency**, meaning the client doesn't need to know where the server is.

3 Server:

- The **Skeleton (server-side handler)** receives the request.

- The actual **server object** executes the method and sends the result back through the ORB.
-

CORBA Architecture

CORBA consists of the following main components:

Component	Description
ORB (Object Request Broker)	The middleware that handles communication between clients and servers.
IDL (Interface Definition Language)	Defines the structure of remote objects and their methods.
Stub (Client-Side Proxy)	Converts method calls into network requests.
Skeleton (Server-Side Handler)	Unpacks client requests and invokes methods on the actual object.
Naming Service	Registers and locates CORBA objects dynamically.

Example: How CORBA is Used?

Suppose a **Java client** wants to communicate with a **C++ server** over the network. The client calls a **remote method** on an object using CORBA. CORBA's **ORB** routes the request to the **server** (written in C++). The server **processes the request** and sends back the response.

Real-World Applications of CORBA

Used in **banking, telecommunications, healthcare, and aerospace** systems where different applications need to communicate reliably.

Example:

Air Traffic Control Systems use CORBA to integrate different radar and communication systems.

Telecom Networks use CORBA to manage call routing across different networks.

Summary

CORBA is a **middleware framework** that enables **language-independent and platform-independent** communication in **distributed systems**.

It uses an **Object Request Broker (ORB)** to mediate between **clients and servers**.

Key Components: ORB, IDL, Stub, Skeleton, Naming Service.

Applications: Used in **large-scale enterprise** and **real-time systems** like banking, telecom, and aerospace.

CORBA is a powerful tool for distributed computing! Let me know if you need more details.

Stub and Skeleton in CORBA

In **CORBA (Common Object Request Broker Architecture)**, **stubs** and **skeletons** are essential components that enable communication between **clients and servers** in a **distributed system**. They act as intermediaries, handling the conversion of **method calls and responses** over the network.

1 Stub (Client-Side)

A **Stub** is a **proxy (local representation)** of the remote object on the client-side.

It provides **the same interface** as the remote object but **forwards method calls to the server** via CORBA's ORB (Object Request Broker).

The stub **marshals (serializes) the method parameters** before sending them to the server.

Example

When the client calls:

```
calculator.add(10, 5);
```

The **stub**:

- Converts 10 and 5 into a **CORBA-compatible request**.
 - Sends it to the **server** via the ORB.
 - Waits for a **response** and then **returns the result** to the client.
-

2 Skeleton (Server-Side)

A **Skeleton** is the **server-side counterpart** of the stub.

It **receives the client's request, unpacks (unmarshals) the parameters**, and **invokes**

the **actual method** on the remote object.
It then **sends the result back** to the client.

Example

The **Skeleton**:

- Receives the **marshaled request** from the ORB.
 - Converts it into a **method call**:
`float result = calculatorImpl.add(10, 5);`
 - Sends the **result (15.0) back** to the client.
-

How Stub & Skeleton Work in CORBA

Client Request Flow

- 1 Client calls `calculator.add(10, 5);`
 - 2 The **Stub** converts the request into a CORBA message and sends it to the ORB.
 - 3 ORB routes the request to the **Skeleton** on the server.
 - 4 The **Skeleton** unmarshals the request and calls `calculatorImpl.add(10, 5);`
 - 5 The **Skeleton** sends the result back to the **Stub**.
 - 6 The **Stub** unmarshals the result and returns it to the client.
-

Generating Stubs & Skeletons in CORBA

When we compile an **IDL file**, CORBA automatically generates the **Stub and Skeleton code**:

```
idlj -fall Calculator.idl
```

This produces:

- **Stub (Client-Side Proxy)**: `CalculatorHelper.java`
 - **Skeleton (Server-Side Handler)**: `CalculatorPOA.java`
-

Summary

Component	Location	Role
Stub	Client-Side	Sends method requests to the server
Skeleton	Server-Side	Receives client requests and invokes the actual method

Stubs and Skeletons enable remote method calls in a CORBA-based distributed application!